

La Programmazione ad Oggetti in Python

Docente: Ambra Demontis

Anno Accademico: 2025 - 2026

Corso di Laurea in Ingegneria Elettronica, Informatica e delle
Telecomunicazioni



University of Cagliari,
Italy

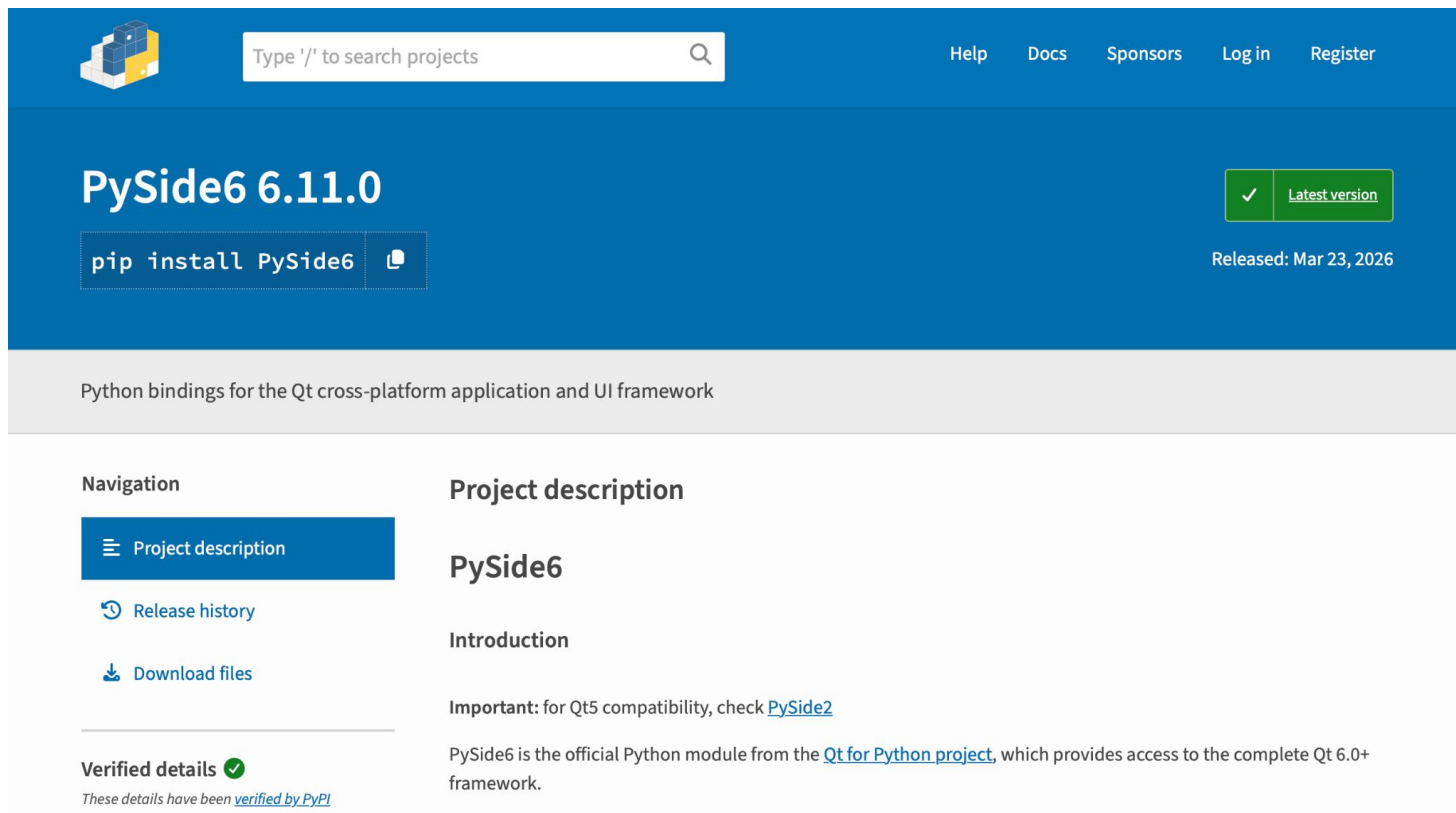
Department of Electrical and
Electronic Engineering



L'utilizzo dei Database

- Installazione PySide6
- Le basi riguardo a QT

Installare il modulo PySide6



The screenshot shows the PySide6 project page on the Python Package Index (PyPI). The page has a blue header with the PySide6 logo, a search bar, and navigation links for Help, Docs, Sponsors, Log in, and Register. The main content area features the project name 'PySide6 6.11.0' in large white text, a green button with a checkmark and 'Latest version', and a green button with the command 'pip install PySide6' and a download icon. Below this, it states 'Released: Mar 23, 2026'. A light gray box contains the description: 'Python bindings for the Qt cross-platform application and UI framework'. The page is divided into two columns. The left column, titled 'Navigation', has a blue button for 'Project description' and links for 'Release history' and 'Download files'. The right column, titled 'Project description', has a sub-header 'PySide6' and an 'Introduction' section. The introduction text states: 'Important: for Qt5 compatibility, check [PySide2](#). PySide6 is the official Python module from the [Qt for Python project](#), which provides access to the complete Qt 6.0+ framework.' At the bottom left, there is a 'Verified details' section with a green checkmark and the text 'These details have been verified by PyPI'.

PySide6 6.11.0

pip install PySide6

Latest version

Released: Mar 23, 2026

Python bindings for the Qt cross-platform application and UI framework

Navigation

- Project description
- Release history
- Download files

Project description

PySide6

Introduction

Important: for Qt5 compatibility, check [PySide2](#)

PySide6 is the official Python module from the [Qt for Python project](#), which provides access to the complete Qt 6.0+ framework.

Verified details ✓
These details have been [verified by PyPI](#)

Testo di Riferimento

Create GUI Applications with Python & Qt6

Autore: Martin Fitzpatrick

Anno pubblicazione: 2025



Creare un'Applicazione Minimale

```
from PySide6.QtWidgets import QApplication, QMainWindow

app = QApplication([])

window = QMainWindow() # crea una finestra
window.show()

app.exec()
```

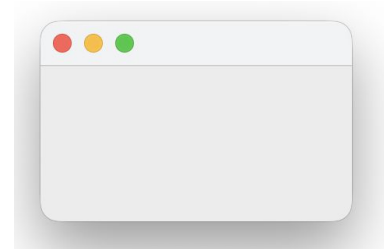
Creare un'Applicazione Minimale

```
from PySide6.QtWidgets import QApplication, QMainWindow

app = QApplication([])

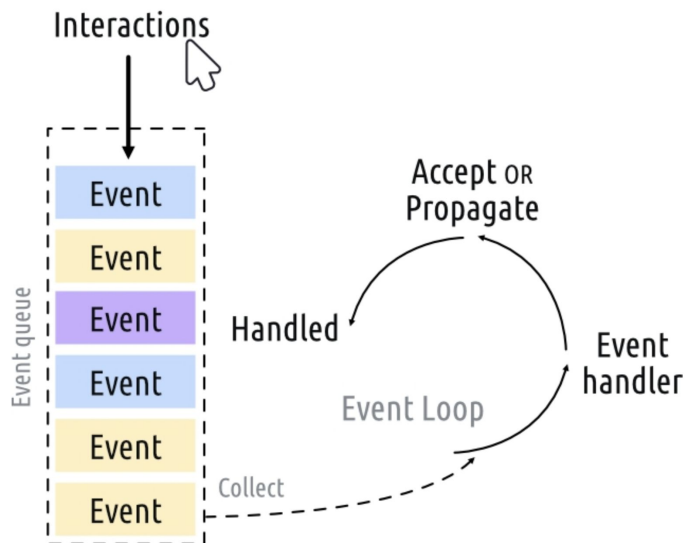
window = QMainWindow() # crea una finestra
window.show()

app.exec()
```



Creare un'Applicazione Minimale

L'oggetto QApplication gestisce il ciclo degli eventi.



Esiste un solo ciclo degli eventi per applicazione

Signal and Slot

Widget componente dell'interfaccia con il quale l'utente può interagire.

Signal notifica emessa da un widget quando qualcosa succede.

Slot ciò che riceve il segnale, per esempio una funzione.

Effettuare Modifiche all'Interfaccia

```
from PySide6.QtWidgets import QApplication, QMainWindow, QPushButton
```

```
class CMainWindow(QMainWindow):
```

```
    def __init__(self):
        super().__init__()

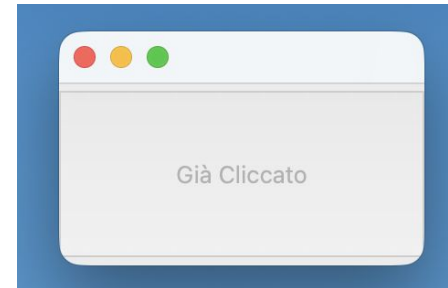
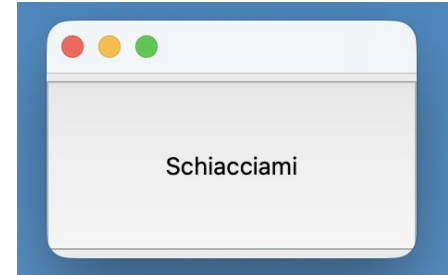
        self.pulsante = QPushButton("Schiacciami")
        self.pulsante.clicked.connect(self.pulsante_cliccato)
        self.setCentralWidget(self.pulsante)
```

```
    def pulsante_cliccato(self):
        self.pulsante.setText("Già Cliccato")
        self.pulsante.setEnabled(False)
```

```
app = QApplication([])
```

```
window = CMainWindow() # crea una finestra
window.show()
```

```
app.exec()
```



Connettere Widget tra Loro

```
from PySide6.QtWidgets import QApplication, QMainWindow, QLabel, QLineEdit, QVBoxLayout, QWidget
```

```
class CMainWindow(QMainWindow):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.label = QLabel()
```

```
        self.input = QLineEdit()
```

```
        self.input.textChanged.connect( self.label.setText) # label gestisce il segnale
```

```
        layout = QVBoxLayout()
```

```
        layout.addWidget( self.input)
```

```
        layout.addWidget( self.label)
```

```
        container = QWidget()
```

```
        container.setLayout(layout)
```

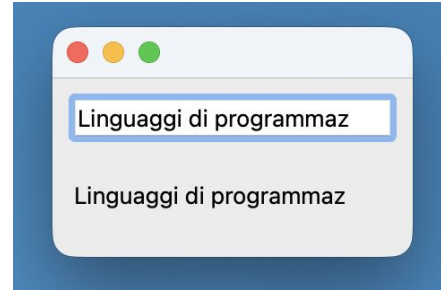
Connettere Widget tra Loro

```
self.setCentralWidget(container)

app = QApplication([])

window = CMainWindow() # crea una finestra
window.show()

app.exec()
```



Nella documentazione si trovano gli slot e i signal es:

<https://doc.qt.io/qt-6/qlabel.html#public-slots>

QCheckBox

```
from PySide6.QtWidgets import QApplication, QMainWindow, QCheckBox
class CMainWindow(QMainWindow):

    def __init__(self):
        super().__init__()

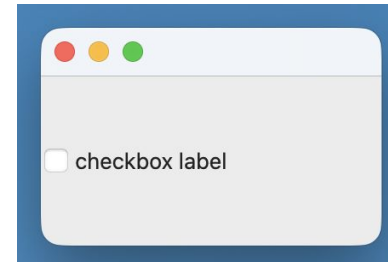
        self.checkbox = QCheckBox("checkbox label")
        self.setCentralWidget(self.checkbox)
        self.checkbox.stateChanged.connect(self.checkbox_state_changed)

    def checkbox_state_changed(self):
        print(self.checkbox.isChecked())

app = QApplication([])

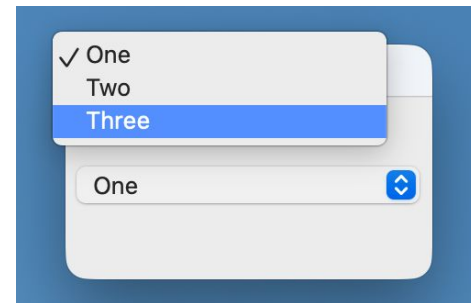
window = CMainWindow() # crea una finestra
window.show()

app.exec()
```



Esercizio: QComboBox

Creare un QComboBox e stampare a schermo il testo selezionato.



Soluzione: QComboBox

```
from PySide6.QtWidgets import QApplication, QMainWindow, QComboBox
class CMainWindow(QMainWindow):
```

```
    def __init__(self):
        super().__init__()

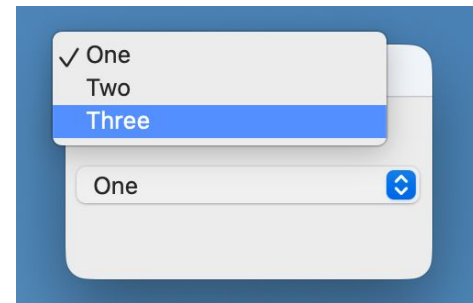
        self.combobox = QComboBox()
        self.combobox.addItem("One")
        self.combobox.addItem("Two")
        self.combobox.addItem("Three")
        self.combobox.currentTextChanged.connect(self.text_changed)
        self.setCentralWidget(self.combobox)
```

```
    def text_changed(self, s):
        print(s)
```

```
app = QApplication([])
```

```
window = CMainWindow() # crea una finestra
window.show()
```

```
app.exec()
```



Layout

Servono ad avere controllo rispetto a dove i widget verranno posizionati.

QGridLayout

0,0	0,1
1,0	1,1

QGridLayout

```
from PySide6.QtWidgets import QApplication, QMainWindow, QPushButton, QGridLayout, QWidget
```

```
class CMainWindow(QMainWindow):
```

```
    def __init__(self):  
        super().__init__()
```

```
        self.pulsante1 = QPushButton("Pulsante 1")  
        self.pulsante2 = QPushButton("Pulsante 2")  
        self.pulsante3 = QPushButton("Pulsante 3")
```

```
        layout = QGridLayout()  
        layout.addWidget(self.pulsante1, 0, 0)  
        layout.addWidget(self.pulsante2, 0, 1)  
        layout.addWidget(self.pulsante3, 1, 1)
```

```
        container = QWidget()  
        container.setLayout(layout)  
        self.setCentralWidget(container)
```


QGridLayout

```
app = QApplication([])

window = CMainWindow() # crea una finestra
window.show()

app.exec()
```



Insert File path

```
from PySide6.QtWidgets import QApplication, QMainWindow, QPushButton, QFileDialog
class CMainWindow(QMainWindow):
```

```
    def __init__(self):
        super().__init__()
```

```
        self.pulsante = QPushButton("Inserisci percorso file")
        self.pulsante.clicked.connect(self.pulsante_cliccato)
        self.setCentralWidget(self.pulsante)
```

Filter default:
All files (*.*)

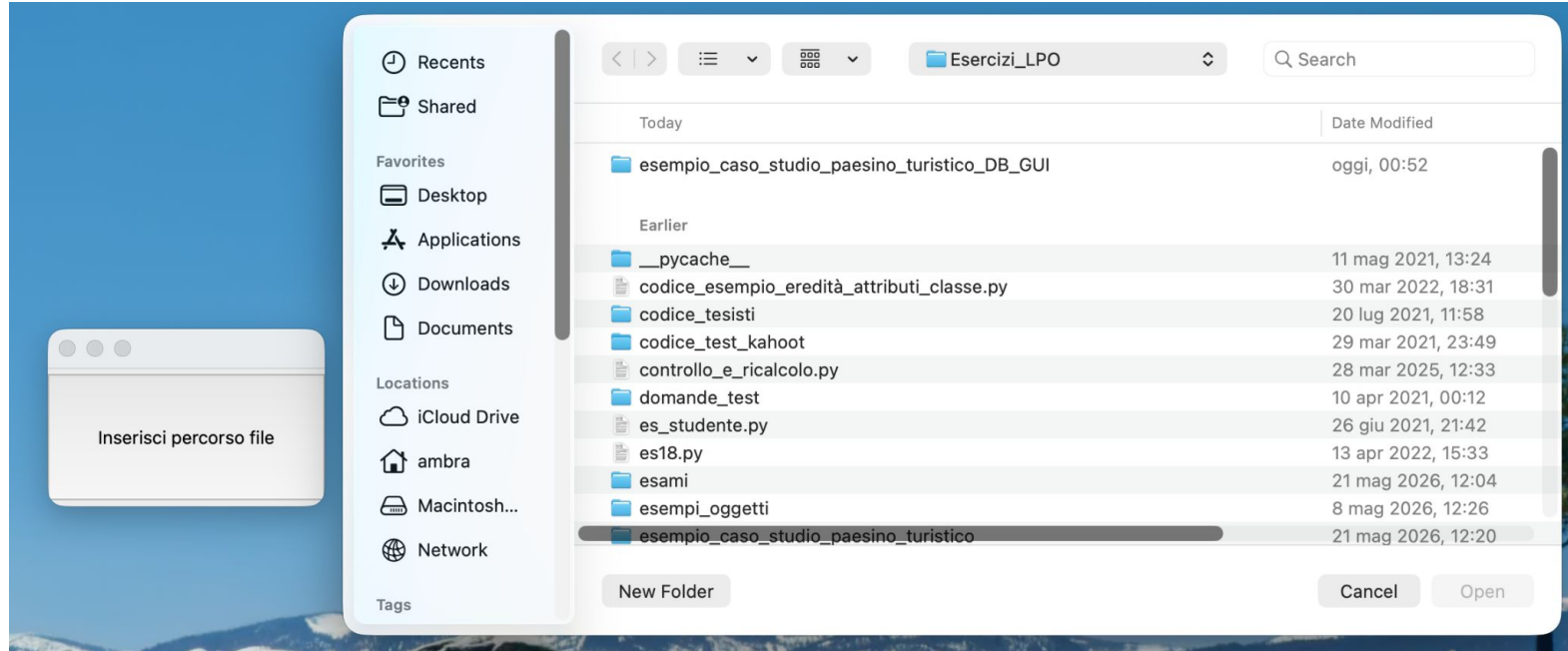


```
    def pulsante_cliccato(self):
        filename, selected_filter = QFileDialog.getOpenFileName(self)
        print(filename)
```

```
app = QApplication([])
```

```
...
```

Insert File path



Insert File path

```
from PySide6.QtWidgets import QApplication, QMainWindow, QPushButton, QFileDialog
class CMainWindow(QMainWindow):

    def __init__(self):
        super().__init__()

        self.pulsante = QPushButton("Inserisci percorso file")
        self.pulsante.clicked.connect(self.pulsante_cliccato)
        self.setCentralWidget(self.pulsante)

    def pulsante_cliccato(self):
        filters = ("Python files (*.py);;Text files (*.txt)")
        filename, selected_filter = QFileDialog.getOpenFileName(self, filter=filters)
        print(filename)

app = QApplication([])

...
```

Dialogs

```
from PySide6.QtWidgets import QApplication, QMainWindow, QPushButton, QMessageBox
from PySide6.QtWidgets import QStyle
```

```
class CMainWindow(QMainWindow):
```

```
    def __init__(self):
        super().__init__()
```

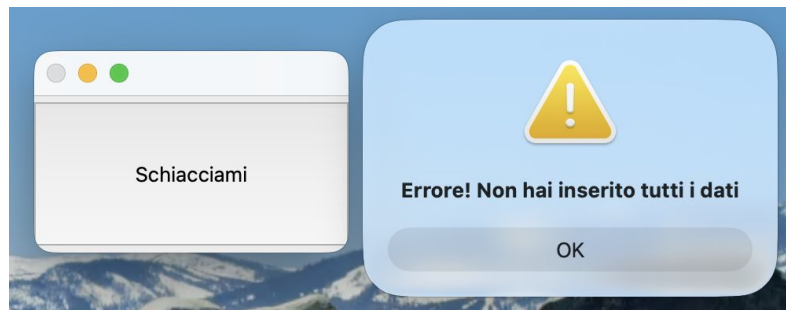
```
        self.pulsante = QPushButton("Schiacciami")
        self.pulsante.clicked.connect(self.pulsante_cliccato)
        self.setCentralWidget(self.pulsante)
```

```
    def pulsante_cliccato(self):
        dialog = QMessageBox(self)
        dialog.setText("Errore! Non hai inserito tutti i dati")
        icon = self.style().standardIcon(QStyle.StandardPixmap.SP_MessageBoxWarning)
        dialog.setIconPixmap(icon.pixmap(64, 64))
        dialog.exec()
```

```
app = QApplication([])
```

```
...
```

Dialogs



Aprire una Seconda Finestra

```
from PySide6.QtWidgets import QApplication, QMainWindow, QPushButton, QFileDialog
```

```
class AnotherWindow(QMainWindow):
```

```
    def __init__(self):  
        super().__init__()  
        self.setWindowTitle("Seconda finestra")
```

Per chiuderla useremo la
funzione `.close()`

```
class CMainWindow(QMainWindow):
```

```
    def __init__(self):  
        super().__init__()  
  
        self.pulsante = QPushButton("Cliccami")  
        self.pulsante.clicked.connect(self.pulsante_cliccato)  
        self.setCentralWidget(self.pulsante)
```

```
    def pulsante_cliccato(self):  
        self.second_window = AnotherWindow()  
        self.second_window.show()
```

...

Passare Dati dalla Seconda Finestra alla Prima

```
from PySide6.QtWidgets import QApplication, QMainWindow, QPushButton, QLabel
from PySide6.QtCore import Signal
```

```
class AnotherWindow(QMainWindow):
```

```
    dataReady = Signal(str)
```



Devono essere attributi di classe

https://doc.qt.io/qtforpython-6.8/tutorials/basictutorial/signals_and_slots.html?utm_source=chatgpt.com

```
    def __init__(self):
        super().__init__()
```

```
        self.pulsante = QPushButton("Close")
        self.pulsante.clicked.connect(self.pulsante_anoth_w_cliccato)
        self.setCentralWidget(self.pulsante)
```

```
    def pulsante_anoth_w_cliccato(self):
        self.dataReady.emit("dati")
```


Passare Dati dalla Seconda Finestra alla Prima

```
class CMainWindow(QMainWindow):  
    def __init__(self):  
        super().__init__()   
  
        self.pulsante = QPushButton("Schiacciami")  
        self.pulsante.clicked.connect(self.pulsante_cliccato)  
        self.setCentralWidget(self.pulsante)  
  
    def pulsante_cliccato(self):  
        self.second_window = AnotherWindow()  
        self.second_window.dataReady.connect(self.gestisci_dati)  
        self.second_window.show()  
  
    def gestisci_dati(self, value):  
        print(value)  
  
app = QApplication([])
```

Qt Designer

E' un'applicazione che viene installata automaticamente.

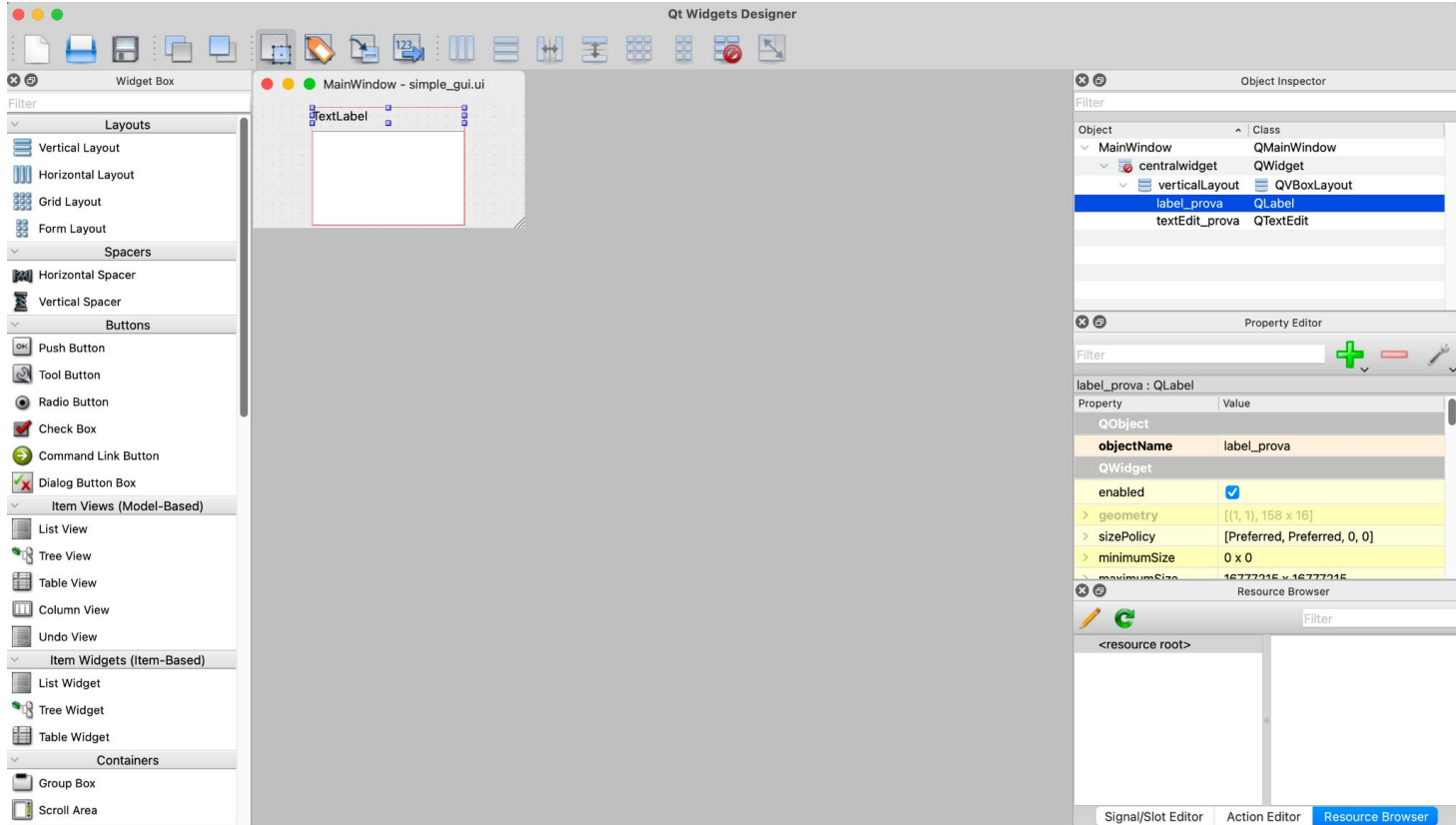
1) Da python:

```
import PySide6, os;  
  
print(os.path.dirname(PySide6.__file__))
```

2) Il path dell'applicazione è quello che otteniamo in quel modo seguito da /Designer.app

3) Da shell: Open path/Designer.app

Qt Designer



Qt Designer

Tradurre il file .ui in .py

Da shell:

```
pyside6-uic <filename>.ui -o <filename>.py
```

Qt Designer

```
# -*- coding: utf-8 -*-
```

```
#####  
## Form generated from reading UI file 'simple_gui.ui'  
##  
## Created by: Qt User Interface Compiler version 6.11.0  
##  
## WARNING! All changes made in this file will be lost when recompiling UI file!  
#####  
from PySide6 import ..
```

```
class Ui_MainWindow(object):  
    def setupUi(self, MainWindow):  
        ...  
        self.label_prova = QLabel(self.verticalLayoutWidget)  
        ...  
        self.lineEdit_prova = QTextEdit(self.verticalLayoutWidget)
```

QtDesigner

```
from PySide6.QtWidgets import QApplication, QMainWindow
from simple_gui import Ui_MainWindow
```

```
class MainWindow(QMainWindow, Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)

        self.lineEdit_prova.textChanged.connect(self.label_prova.setText)
```

Dobbiamo necessariamente ereditare da entrambe

```
app = QApplication([])
```

```
window = MainWindow() # crea una finestra
window.show()
```

```
app.exec()
```

